Simple To-Do List Project Report

Python CLI + CRUD + GitHub + Render Deployment Preparation

Prepared: 27 September 2026

1. Project Overview

The project is a command-line to-do application designed to demonstrate Python fundamentals: lists, loops, conditionals, functions, menu-driven programming and CRUD operations. The primary deliverable is the CLI application. A lightweight Flask web adapter is included so the same CRUD logic can be demonstrated through a Render Web Service.

2. Objectives

• Store tasks in a Python list.

• Create separate functions for CRUD operations.

• Provide a clear menu-driven command-line interface.

• Handle invalid menu choices and invalid task IDs.

• Test the core CRUD logic with Python unittest.

• Provide GitHub and Render deployment configuration.

3. Functional Features

Feature	Implementation
Create	Add a new task with a generated numeric ID.
Read	Display all tasks with Pending/Done status.
Update	Change task title and completed status.
Delete	Remove a task by ID.
Validation	Reject empty titles and invalid numeric IDs.
Menu handling	Reject unsupported menu options without crashing.

4. Technologies Used

• Python 3.10+ — core programming language.

• Flask 3.1.3 — optional web adapter for deployment demonstration.

• Gunicorn 26.2.0 — production WSGI server used by Render.

• Git/GitHub — source control and public repository submission.

• Render — cloud deployment platform for the web demonstration.

• HTML/CSS — simple browser interface.

• unittest — automated tests for CRUD logic.

5. Architecture

The project separates the reusable task logic from the user interfaces. **todo_core.py** owns the task list and CRUD functions. **todo_cli.py** imports those functions and provides the required command-line menu. **web_app.py** imports

the same functions and exposes a small HTML interface plus JSON API for Render. This separation keeps the CRUD logic focused and testable.

6. Local Execution

```
py -3 -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
python -m unittest discover -s . -p "test_*.py" -v
python todo_cli.py
```

The included unit test suite covers successful Create/Read/Update/Delete behavior, invalid empty titles, and missing task IDs.

7. Render Deployment

The repository contains a root-level **render.yaml** Blueprint. It defines one Python web service with a build command, Gunicorn start command and /health check. Render Web Services must listen on 0.0.0.0 and use the PORT environment variable; the project follows that requirement.

```
Build command:
pip install -r requirements.txt

Start command:
gunicorn web_app:app --bind 0.0.0.0:$PORT --workers 1

Health check:
 /health
```

The single-worker setting is intentional because the demonstration stores tasks in an in-memory Python list. A multi-worker deployment could have separate memory state in different processes.

8. Data Persistence Limitation

This is an educational CRUD project and intentionally uses a list rather than a database. The Render demo therefore does not provide durable task storage. Render's default service filesystem is ephemeral, and free web services can spin down. A production version should use a managed database or another durable datastore if persistent tasks are required.

9. GitHub Submission

```
git init
git add .
git commit -m "Initial commit - Simple To-Do List"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/simple-todo-list.git
git push -u origin main
```

The repository should contain the Python source, tests, requirements, render.yaml, README, deployment guide, submission checklist, sample execution, LinkedIn draft, license and report.

10. Sample Execution Evidence

The package includes SAMPLE_EXECUTION.txt with representative output. For the actual internship submission, capture a screenshot from your own PowerShell run showing Add, View, Update, Remove and invalid-menu handling. The representative text is not a substitute for actual evidence.

11. Rollback Evidence

A real Render rollback cannot be performed from this package because it requires access to the student's Render account and an actual deployment history. The deployment guide therefore provides the exact evidence workflow

without fabricating a rollback screenshot or claiming that a rollback has already happened. After the first successful deployment, create a second deployment, capture the Deploys history, perform the required rollback, and capture the resulting status.

12. Interview Questions and Answers

Why are lists useful?

They store multiple task records in an ordered, mutable collection and are simple to iterate over.

What does CRUD mean?

Create, Read, Update and Delete — the four basic operations implemented by the application.

Why separate operations into functions?

Focused functions improve readability, reuse, testing and maintenance.

13. Submission Checklist

■ Run the CLI locally and test all CRUD operations.

■ Run the unit tests and confirm they pass.

■ Push the repository to GitHub and verify the public link.

■ Deploy the Render Blueprint and verify /, /health and /api/tasks.

■ Capture actual terminal, GitHub and Render screenshots.

■ Perform and capture the required rollback evidence.

■ Submit the GitHub repository link through the internship portal.

■ Share the completed project on LinkedIn using the included draft.

14. Reference Documentation

https://render.com/docs/web-services

https://render.com/docs/deploys

https://render.com/docs/blueprints

https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository

https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github

Important: account-specific deployment results, live URLs, screenshots and rollback history must be recorded from the actual GitHub/Render accounts used for submission.